Serviço de Processamento de Arquivos com gRPC

Laboratório 7 - C++ + Python + Protocol Buffers + Docker

Identificação	Informação
Projeto	Serviço de Processamento de Arquivos com gRPC
Disciplina	Computação Distribuída
Instituição	Universidade Presbiteriana Mackenzie
Professores do enunciado	Prof. Alcides / Prof. Mário
Nome	Ryan Ledo
RA	10352727
Data do relatório	02/11/2026

1. Introdução e Fundamentação Teórica

1.1 Objetivo

Neste laboratório, organizei um serviço de processamento de arquivos com servidor C++ e cliente Python. Implementei a compressão de PDF da Entrega 1 e as três operações da Entrega 2: extração de texto, conversão de imagem e redimensionamento. Cada programa executa em um container e utiliza o mesmo contrato Protobuf.

1.2 gRPC, Protobuf e streaming

O contrato define as mensagens e os métodos disponíveis. O protoc gera as classes C++ e Python usadas na comunicação. Para os arquivos, utilizei streaming bidirecional: envio os metadados no primeiro bloco e o conteúdo nos seguintes; depois do processamento, recebo o resultado em blocos. O servidor grava o upload em disco antes de iniciar a ferramenta externa. O método Health é unário e retorna SERVING.

O exemplo do enunciado coloca stream dentro das mensagens, mas a sintaxe válida utiliza stream na assinatura do RPC. Ajustei esse ponto para gerar código compilável. Uma falha retorna um status gRPC; o cliente só publica o arquivo de saída quando o stream termina com sucesso.

2. Ambiente e Organização

2.1 Ambiente observado

Componente	Versão ou configuração
Host	Windows, Docker Desktop com containers Linux
Docker Engine / Compose	29.7.2 / 5.5.0
Base do servidor	Debian bookworm-slim, C++17

Componente	Versão ou configuração
Compilador / CMake	GCC 12.2.0 / CMake 3.25.1
gRPC C++ / Protobuf C++	1.51.1 / 3.21.12
Cliente	Imagem Python 3.12-slim-bookworm; grpcio 1.76.0
Ghostscript / Poppler	10.00.0 / 22.12.0
ImageMagick	6.9.11-60 Q16

2.2 Serviços e isolamento

Separei servidor e cliente em imagens próprias. O build do servidor gera os stubs e compila o executável; a imagem final contém apenas as bibliotecas e ferramentas necessárias à execução. O servidor roda com UID 10001, limite de 512 MiB e 2 CPUs. O cliente utiliza server:50051 pela rede do Compose. A publicação no host fica restrita a 127.0.0.1:50051.

Mantive os resultados em arquivos/ e o log em evidencias/server.log, por bind mounts. Os temporários usam diretórios exclusivos em /tmp dentro do servidor e são removidos ao terminar a requisição. O cliente aguarda o canal gRPC estar disponível, pois iniciar o container não garante que o serviço já esteja pronto.

2.3 Organização dos entregáveis

proto/file_processor.proto contém o contrato; server_cpp/server.cc implementa FileProcessorServiceImpl; client_python/client.py fornece a interface de linha de comando. client_python/testes.py gera exemplos e valida as respostas. Os Dockerfiles e docker-compose.yml reproduzem o ambiente. README.md contém os comandos. Este Markdown é a fonte do relatório e não é reescrito pelo gerador de PDF.

3. Implementação das Entregas

3.1 Entrega 1 - CompressPDF

Recebo o PDF em blocos de 64 KiB e verifico o cabeçalho e a estrutura com pdfinfo. Depois executo Ghostscript com pdfwrite, compatibilidade 1.4 e perfil /ebook, conforme o enunciado. A resposta contém o PDF processado. O sucesso indica que o arquivo foi processado; não garante redução de tamanho para qualquer entrada.

3.2 Entrega 2 - ConvertToTXT

Utilizo pdftotext para extrair o texto do PDF validado. O conteúdo é transmitido ao cliente e salvo em disco. Um PDF sem texto pode produzir TXT vazio; por isso, envio um cabeçalho de resposta mesmo quando não existem bytes de texto. Não implementei OCR para documentos digitalizados.

3.3 Entrega 2 - ConvertImageFormat e ResizeImage

Utilizo convert do ImageMagick para as duas operações. A conversão aceita png, jpg e webp. O redimensionamento recebe largura e altura entre 1 e 8192 e preserva a proporção, como no comando do enunciado. A saída desse método é PNG. Em arquivos com múltiplos frames, processo o primeiro.

3.4 Erros, execução externa e logs

Passei os argumentos das ferramentas por posix_spawnp, sem montar comandos de shell com valores enviados pelo cliente. O limite de upload é 32 MiB; cada comando externo tem limite de 30 segundos. O cliente usa deadline de 60 segundos. O PDF pode passar por validação e processamento separados, cada um com seu limite.

Entradas inválidas retornam INVALID_ARGUMENT; upload acima do limite retorna RESOURCE_EXHAUSTED. Falhas internas e limites de tempo têm status próprios. O cliente usa um arquivo temporário e só substitui o destino depois de concluir a RPC, preservando um resultado anterior se ocorrer erro.

O log registra método, arquivo, timestamps UTC, status, mensagem e diagnóstico da ferramenta. O timestamp da linha marca o resultado da operação; requested_at registra o início da requisição na versão final. Arquivo ausente no cliente falha antes de enviar a RPC e, portanto, não produz uma requisição no log do servidor. Os logs são cumulativos, inclusive durante a depuração.

4. Execução e Evidências

4.1 Origem dos registros e inicialização

Observação: Deixei a saída do terminal em texto copiado, pois ela é muito grande e um print não consegue mostrar toda a saída. Dessa forma, mantive o conteúdo completo para facilitar a leitura e a conferência dos resultados.

Executei novamente os códigos e registrei a saída completa de cada comando, incluindo stdout, stderr e código de saída. A coleta começou em 2026-11-02T20:41:04.194084+00:00. Substituí os prints pelas transcrições integrais do terminal. Os blocos abaixo não omitem linhas; as quebras de linhas longas no PDF servem somente à diagramação. O caractere de controle de fim de página do TXT aparece como \f para ficar visível.

Os arquivos originais de cada comando, o índice registros.json e terminal-completo.txt estão em evidencias/terminal-20261102T204104Z/. O log desta rodada contém todas as linhas do servidor desde o início da coleta. O arquivo evidencias/server.log continua acumulando também as rodadas anteriores. O servidor é contínuo: os comandos de inicialização e consulta de logs registram sua execução sem encerrar o processo.

Todos os comandos abaixo foram executados na pasta Lab_7. Os casos de erro esperados retornaram código 1; os demais retornaram 0.

servidor

```
$ docker compose up -d server
 Container laboratorio7-server-1 Running

[codigo de saida: 0]
```

containers

```
$ docker compose ps
NAME                    IMAGE                 COMMAND    SERVICE   CREATED
STATUS          PORTS
laboratorio7-server-1   laboratorio7-server   "server"   server    13 minutes ago
Up 13 minutes   127.0.0.1:50051->50051/tcp

[codigo de saida: 0]
```

health

```
$ docker compose run --rm -T client health
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-dce54234eea4 Creating
 Container laboratorio7-client-run-dce54234eea4 Created
SERVING

[codigo de saida: 0]
```

4.2 Entrega 1 - Compressão e verificação dos PDFs

O PDF com imagem passou de 729159 para 585050 bytes. O PDF somente com texto passou de 1441 para 2833 bytes. A leitura dos resultados confirmou uma página e a preservação do texto nos dois casos.

compressao-entrada

```
$ docker compose run --rm -T client compress /data/entrada.pdf /data/terminal-
entrada.pdf
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-8eb0957c5666 Creating
 Container laboratorio7-client-run-8eb0957c5666 Created
OK CompressPDF: /data/terminal-entrada.pdf (585050 bytes)

[codigo de saida: 0]
```

compressao-texto

```
$ docker compose run --rm -T client compress /data/texto.pdf /data/terminal-
texto.pdf
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-dbd65ce51ba3 Creating
 Container laboratorio7-client-run-dbd65ce51ba3 Created
OK CompressPDF: /data/terminal-texto.pdf (2833 bytes)
```

```
[codigo de saida: 0]
```

verificacao-pdfs

```
$ docker compose run --rm -T --entrypoint python client -c "from pathlib import
Path; from pypdf import PdfReader; [(print(f'{p.name}: {p.stat().st_size} bytes;
paginas={len(PdfReader(p).pages)}; texto=
{PdfReader(p).pages[0].extract_text()!r}')) for p in
[Path('/data/entrada.pdf'),Path('/data/terminal-
entrada.pdf'),Path('/data/texto.pdf'),Path('/data/terminal-texto.pdf')]]"
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-388b3459fcb8 Creating
 Container laboratorio7-client-run-388b3459fcb8 Created
entrada.pdf: 729159 bytes; paginas=1; texto='Laboratorio 7 - gRPC: teste de
processamento de arquivos.\n'
terminal-entrada.pdf: 585050 bytes; paginas=1; texto='Laboratorio 7 - gRPC: teste
de processamento de arquivos.\n'
texto.pdf: 1441 bytes; paginas=1; texto='Segundo arquivo PDF para teste.\n'
terminal-texto.pdf: 2833 bytes; paginas=1; texto='Segundo arquivo PDF para teste.'

[codigo de saida: 0]
```

4.3 Entrega 2 - Extração e leitura completa do texto

Executei ConvertToTXT sobre os dois PDFs e li integralmente os arquivos produzidos. Os bytes e o conteúdo aparecem nas saídas abaixo.

extracao-entrada

```
$ docker compose run --rm -T client text /data/entrada.pdf /data/terminal-
entrada.txt
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-642d64f7ea68 Creating
 Container laboratorio7-client-run-642d64f7ea68 Created
OK ConvertToTXT: /data/terminal-entrada.txt (60 bytes)

[codigo de saida: 0]
```

conteudo-entrada

```
$ docker compose run --rm -T --entrypoint python client -c "from pathlib import
Path; import sys; sys.stdout.write(Path('/data/terminal-
entrada.txt').read_text())"
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-f72fd298827a Creating
```

```
 Container laboratorio7-client-run-f72fd298827a Created
Laboratorio 7 - gRPC: teste de processamento de arquivos.

\f
[codigo de saida: 0]
```

extracao-texto

```
$ docker compose run --rm -T client text /data/texto.pdf /data/terminal-texto.txt
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-eb5a34139cfd Creating
 Container laboratorio7-client-run-eb5a34139cfd Created
OK ConvertToTXT: /data/terminal-texto.txt (34 bytes)

[codigo de saida: 0]
```

conteudo-texto

```
$ docker compose run --rm -T --entrypoint python client -c "from pathlib import
Path; import sys; sys.stdout.write(Path('/data/terminal-texto.txt').read_text())"
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-84e17ede5a16 Creating
 Container laboratorio7-client-run-84e17ede5a16 Created
Segundo arquivo PDF para teste.

\f
[codigo de saida: 0]
```

4.4 Entrega 2 - Conversão de formatos

Executei as conversões para PNG, JPEG e WEBP. A inspeção leu o formato real e as dimensões dos arquivos, confirmando 1200x900 nos três resultados.

conversao-png

```
$ docker compose run --rm -T client convert /data/imagem.jpg /data/terminal-
convertida.png --format png
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-bb1cc996377a Creating
 Container laboratorio7-client-run-bb1cc996377a Created
OK ConvertImageFormat: /data/terminal-convertida.png (1082780 bytes)

[codigo de saida: 0]
```

conversao-jpg

```
$ docker compose run --rm -T client convert /data/imagem.jpg /data/terminal-
convertida.jpg --format jpg
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-8d26a6bfdf24 Creating
 Container laboratorio7-client-run-8d26a6bfdf24 Created
OK ConvertImageFormat: /data/terminal-convertida.jpg (558244 bytes)

[codigo de saida: 0]
```

conversao-webp

```
$ docker compose run --rm -T client convert /data/imagem.jpg /data/terminal-
convertida.webp --format webp
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-3566c15a2dfa Creating
 Container laboratorio7-client-run-3566c15a2dfa Created
OK ConvertImageFormat: /data/terminal-convertida.webp (565646 bytes)

[codigo de saida: 0]
```

verificacao-formatos

```
$ docker compose run --rm -T --entrypoint python client -c "from PIL import Image;
from pathlib import Path; [(print(p.name, Image.open(p).format,
Image.open(p).size)) for p in sorted(Path('/data').glob('terminal-
convertida.*'))]"
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-46965e306bfc Creating
 Container laboratorio7-client-run-46965e306bfc Created
terminal-convertida.jpg JPEG (1200, 900)
terminal-convertida.png PNG (1200, 900)
terminal-convertida.webp WEBP (1200, 900)

[codigo de saida: 0]
```

4.5 Entrega 2 - Redimensionamento

O caso 400x300 produziu essas dimensões. O limite 200x200 produziu 200x150, preservando a proporção da entrada.

redimensionamento-400x300

```
$ docker compose run --rm -T client resize /data/imagem.png /data/terminal-menor-
400.png --width 400 --height 300
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-4c8f00c528c1 Creating
 Container laboratorio7-client-run-4c8f00c528c1 Created
OK ResizeImage: /data/terminal-menor-400.png (98944 bytes)

[codigo de saida: 0]
```

redimensionamento-200x200

```
$ docker compose run --rm -T client resize /data/imagem.png /data/terminal-menor-
200.png --width 200 --height 200
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-5600405e7877 Creating
 Container laboratorio7-client-run-5600405e7877 Created
OK ResizeImage: /data/terminal-menor-200.png (21350 bytes)

[codigo de saida: 0]
```

verificacao-dimensoes

```
$ docker compose run --rm -T --entrypoint python client -c "from PIL import Image;
from pathlib import Path; [(print(p.name, Image.open(p).format,
Image.open(p).size)) for p in sorted(Path('/data').glob('terminal-menor-*.png'))]"
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-3f9613df5cfe Creating
 Container laboratorio7-client-run-3f9613df5cfe Created
terminal-menor-200.png PNG (200, 150)
terminal-menor-400.png PNG (400, 300)

[codigo de saida: 0]
```

4.6 Tratamento de erros pela linha de comando

Executei os casos de formato não permitido, arquivo ausente, PDF corrompido, PDF vazio e dimensão inválida. Cada execução retornou código 1. O arquivo ausente falhou localmente antes do envio de uma RPC.

erro-formato

```
$ docker compose run --rm -T client convert /data/imagem.png /data/terminal-
erro.bin --format exe
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-54baa64d5117 Creating
```

```
 Container laboratorio7-client-run-54baa64d5117 Created
Erro gRPC INVALID_ARGUMENT: Formato permitido: png, jpg ou webp


[codigo de saida: 1]
```

erro-ausente

```
$ docker compose run --rm -T client compress /data/ausente.pdf /data/terminal-
erro.pdf
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-9142014649fd Creating
 Container laboratorio7-client-run-9142014649fd Created
Erro local: [Errno 2] No such file or directory: '/data/ausente.pdf'


[codigo de saida: 1]
```

erro-pdf

```
$ docker compose run --rm -T client compress /data/invalido.pdf /data/terminal-
erro.pdf
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-c2b7617f5b35 Creating
 Container laboratorio7-client-run-c2b7617f5b35 Created
Erro gRPC INVALID_ARGUMENT: PDF invalido ou validacao excedeu prazo


[codigo de saida: 1]
```

erro-vazio

```
$ docker compose run --rm -T client text /data/vazio.pdf /data/terminal-erro.txt
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-0709e2089faa Creating
 Container laboratorio7-client-run-0709e2089faa Created
Erro gRPC INVALID_ARGUMENT: Arquivo vazio


[codigo de saida: 1]
```

erro-dimensao

```
$ docker compose run --rm -T client resize /data/imagem.png /data/terminal-
erro.png --width 0 --height 100
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-6eb50ec3a568 Creating
 Container laboratorio7-client-run-6eb50ec3a568 Created
Erro gRPC INVALID_ARGUMENT: Dimensoes devem estar entre 1 e 8192


[codigo de saida: 1]
```

4.7 Bateria completa de testes e concorrência

A saída integral abaixo registra os 22 testes aprovados, incluindo quatro chamadas simultâneas, validação do protocolo, limite de upload e preservação do arquivo de saída após erro. Não foram realizados testes de carga prolongada, cancelamento em andamento ou exaustão de recursos.

testes-completos

```
$ docker compose run --rm -T --entrypoint python client /app/testes.py
 Container laboratorio7-server-1 Running
 Container laboratorio7-client-run-36731325150b Creating
 Container laboratorio7-client-run-36731325150b Created
{"caso": "RPC unario Health", "aprovado": true, "detalhes": "SERVING"}
{"caso": "CompressPDF entrada.pdf", "aprovado": true, "detalhes":
{"entrada_bytes": 729159, "saida_bytes": 585050}}
{"caso": "CompressPDF texto.pdf", "aprovado": true, "detalhes": {"entrada_bytes":
1441, "saida_bytes": 2833}}
{"caso": "ConvertToTXT entrada.pdf", "aprovado": true, "detalhes": {"texto":
"Laboratorio 7 - gRPC: teste de processamento de arquivos."}}
{"caso": "ConvertToTXT texto.pdf", "aprovado": true, "detalhes": {"texto":
"Segundo arquivo PDF para teste."}}
{"caso": "ConvertImageFormat png", "aprovado": true, "detalhes": {"formato":
"PNG", "dimensoes": [1200, 900]}}
{"caso": "ConvertImageFormat jpg", "aprovado": true, "detalhes": {"formato":
"JPEG", "dimensoes": [1200, 900]}}
{"caso": "ConvertImageFormat webp", "aprovado": true, "detalhes": {"formato":
"WEBP", "dimensoes": [1200, 900]}}
{"caso": "ResizeImage 400x300", "aprovado": true, "detalhes": {"limites": [400,
300], "dimensoes": [400, 300]}}
{"caso": "ResizeImage preserva proporcao 200x200", "aprovado": true, "detalhes":
{"limites": [200, 200], "dimensoes": [200, 150]}}
{"caso": "PDF corrompido", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "PDF invalido ou validacao excedeu prazo"}}
{"caso": "PDF vazio", "aprovado": true, "detalhes": {"status": "INVALID_ARGUMENT",
"mensagem": "Arquivo vazio"}}
{"caso": "Imagem como PDF", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Conteudo nao e PDF"}}
{"caso": "Formato invalido", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Formato permitido: png, jpg ou webp"}}
```

```
{"caso": "Dimensao invalida", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Dimensoes devem estar entre 1 e 8192"}}
{"caso": "Imagem corrompida", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Ferramenta rejeitou o arquivo; consulte
server.log"}}
{"caso": "Arquivo nao encontrado", "aprovado": true, "detalhes":
"FileNotFoundError local; nenhuma RPC enviada"}
{"caso": "Stream sem metadata", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Primeiro bloco deve conter metadata"}}
{"caso": "Nome com caminho", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Nome de arquivo invalido"}}
{"caso": "Metadata repetida", "aprovado": true, "detalhes": {"status":
"INVALID_ARGUMENT", "mensagem": "Metadata repetida ou bloco sem payload"}}
{"caso": "Limite de upload 32 MiB", "aprovado": true, "detalhes": {"status":
"RESOURCE_EXHAUSTED", "mensagem": "Limite de upload: 32 MiB"}}
{"caso": "Concorrencia", "aprovado": true, "detalhes": "4 chamadas com mesmo nome
de entrada, resultados independentes"}
APROVADOS: 22 testes

[codigo de saida: 0]
```

4.8 Limpeza e log completo do servidor nesta coleta

A consulta dos diretórios temporários terminou com código 0 e sem linhas de resultado, indicando que não havia diretórios lab7-* naquele momento. A seguir, incluí toda a saída do log do servidor desde o início desta coleta, sem selecionar apenas as últimas linhas.

temporarios

```
$ docker compose exec -T server find /tmp -maxdepth 1 -name lab7-*

[codigo de saida: 0]
```

logs-servidor

```
$ docker compose logs --no-color --since 2026-11-02T20:41:04.194084+00:00 server
server-1  | 2026-11-02T20:41:06Z service=Health requested_at=2026-11-02T20:41:06Z
file="" status=0 message="sucesso" detail=""
server-1  | 2026-11-02T20:41:09Z service=Health requested_at=2026-11-02T20:41:09Z
file="" status=0 message="sucesso" detail=""
server-1  | 2026-11-02T20:41:10Z service=CompressPDF requested_at=2026-11-
02T20:41:09Z file="entrada.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:10Z service=CompressPDF requested_at=2026-11-
02T20:41:10Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:10Z service=ConvertToTXT requested_at=2026-11-
02T20:41:10Z file="entrada.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:10Z service=ConvertToTXT requested_at=2026-11-
02T20:41:10Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
```

```
server-1  | 2026-11-02T20:41:11Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:10Z file="imagem.jpg" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:11Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:11Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:11Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:11Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=ResizeImage requested_at=2026-11-
02T20:41:11Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=ResizeImage requested_at=2026-11-
02T20:41:12Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=CompressPDF requested_at=2026-11-
02T20:41:12Z file="invalido.pdf" status=3 message="PDF invalido ou validacao
excedeu prazo" detail="pdfinfo exit=1 Syntax Error: Couldn't find trailer
dictionary Syntax Error: Couldn't find trailer dictionary Syntax Error: Couldn't
read xref table "
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="vazio.pdf" status=3 message="Arquivo vazio" detail=""
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="imagem.png" status=3 message="Conteudo nao e PDF" detail=""
server-1  | 2026-11-02T20:41:12Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:12Z file="imagem.png" status=3 message="Formato permitido: png, jpg ou
webp" detail=""
server-1  | 2026-11-02T20:41:12Z service=ResizeImage requested_at=2026-11-
02T20:41:12Z file="imagem.png" status=3 message="Dimensoes devem estar entre 1 e
8192" detail=""
server-1  | 2026-11-02T20:41:12Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:12Z file="invalido.pdf" status=3 message="Ferramenta rejeitou o arquivo;
consulte server.log" detail="exit=1 convert-im6.q16: attempt to perform an
operation not allowed by the security policy `PDF' @
error/constitute.c/IsCoderAuthorized/426. convert-im6.q16: no images defined
`/tmp/lab7-EynHg8/output.png' @ error/convert.c/ConvertImageCommand/3234. "
server-1  | 2026-11-02T20:41:12Z service=CompressPDF requested_at=2026-11-
02T20:41:12Z file="" status=3 message="Primeiro bloco deve conter metadata"
detail=""
server-1  | 2026-11-02T20:41:12Z service=CompressPDF requested_at=2026-11-
02T20:41:12Z file="../escape.pdf" status=3 message="Nome de arquivo invalido"
detail=""
server-1  | 2026-11-02T20:41:12Z service=CompressPDF requested_at=2026-11-
02T20:41:12Z file="a.pdf" status=3 message="Metadata repetida ou bloco sem
payload" detail=""
server-1  | 2026-11-02T20:41:12Z service=CompressPDF requested_at=2026-11-
02T20:41:12Z file="grande.pdf" status=8 message="Limite de upload: 32 MiB"
detail=""
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:12Z service=ConvertToTXT requested_at=2026-11-
02T20:41:12Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:14Z service=CompressPDF requested_at=2026-11-
02T20:41:14Z file="entrada.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:16Z service=CompressPDF requested_at=2026-11-
```

```
02T20:41:15Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:19Z service=ConvertToTXT requested_at=2026-11-
02T20:41:19Z file="entrada.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:21Z service=ConvertToTXT requested_at=2026-11-
02T20:41:21Z file="texto.pdf" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:25Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:24Z file="imagem.jpg" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:26Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:26Z file="imagem.jpg" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:28Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:28Z file="imagem.jpg" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:31Z service=ResizeImage requested_at=2026-11-
02T20:41:31Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:33Z service=ResizeImage requested_at=2026-11-
02T20:41:33Z file="imagem.png" status=0 message="sucesso" detail="exit=0 "
server-1  | 2026-11-02T20:41:36Z service=ConvertImageFormat requested_at=2026-11-
02T20:41:36Z file="imagem.png" status=3 message="Formato permitido: png, jpg ou
webp" detail=""
server-1  | 2026-11-02T20:41:39Z service=CompressPDF requested_at=2026-11-
02T20:41:39Z file="invalido.pdf" status=3 message="PDF invalido ou validacao
excedeu prazo" detail="pdfinfo exit=1 Syntax Error: Couldn't find trailer
dictionary Syntax Error: Couldn't find trailer dictionary Syntax Error: Couldn't
read xref table "
server-1  | 2026-11-02T20:41:41Z service=ConvertToTXT requested_at=2026-11-
02T20:41:41Z file="vazio.pdf" status=3 message="Arquivo vazio" detail=""
server-1  | 2026-11-02T20:41:43Z service=ResizeImage requested_at=2026-11-
02T20:41:43Z file="imagem.png" status=3 message="Dimensoes devem estar entre 1 e
8192" detail=""

[codigo de saida: 0]
```

5. Análise dos Resultados

Nos testes, observei comunicação entre C++ e Python com o contrato gerado do mesmo arquivo .proto. O PDF com imagem diminuiu aproximadamente 19,76%; o PDF pequeno de texto aumentou, pois o processamento também altera a estrutura e inclui metadados. Esse resultado mostra por que o sucesso da compressão não pode ser definido somente pela comparação de tamanhos.

As dimensões e os formatos foram verificados nos arquivos recebidos, e não apenas pela extensão. O caso 200x200 confirmou a preservação da proporção. As chamadas simultâneas com o mesmo nome de entrada não misturaram os resultados, usando diretórios temporários independentes.

Os testes representam uma carga controlada no Docker Desktop, sem distribuição entre máquinas físicas. O protótipo não utiliza autenticação ou TLS. Os limites de tamanho, memória e tempo reduzem riscos do experimento, mas não substituem isolamento adicional em produção. O cancelamento é verificado após receber e processar; uma ferramenta já iniciada pode continuar até terminar ou atingir seu limite.

6. Dificuldades Encontradas e Soluções

Dificuldade observada	Solução adotada
Exemplo de stream em campo Protobuf não compilável	Colocar stream nas assinaturas RPC e separar metadata de bytes
Docker Desktop inicialmente parado	Iniciar o Docker e verificar o container e a RPC Health
Ghostscript aceitou o PDF corrompido no primeiro teste	Validar a estrutura com pdfinfo antes de processar e repetir os testes
Captura de terminal bloqueada pelo controle do computador	Registrar novas execuções e incluir as saídas completas no lugar dos prints

7. Declaração e Análise do Uso de IA

Utilizei o ChatGPT como apoio para interpretar o enunciado, montar o laboratório, gerar e depurar os códigos C++ e Python e organizar os testes e as evidências.

8. Conclusão

Implementei as duas entregas e executei os quatro serviços em containers. A rodada final aprovou 22 testes, incluindo validação dos resultados, tratamento de erros e concorrência. O código, os arquivos processados e os logs estão disponíveis para reprodução. Registrei as saídas completas dos comandos e do servidor no lugar dos prints. Essas evidências documentam a execução realizada e não substituem a demonstração em laboratório.

9. Referências

- Material da disciplina: Serviço de Processamento de Arquivos com gRPC, Prof. Alcides / Prof. Mário, texto fornecido do PDF, páginas 1 a 9.
- Modelo de relatório fornecido: Lab_6/relatorio.md.
- gRPC, tutorial C++: https://grpc.io/docs/languages/cpp/basics/
- Protocol Buffers, especificação proto3: https://protobuf.dev/reference/protobuf/proto3-spec/